

BIT1123 / BISE2093 / DIT1113
OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 2 (GROUP) – 20%

Library Management System

Full Name	Student ID	Class Code	Program	NRIC / Passport No.

(Fill in the details of every group member above before submission.)

GitHub Repository: [paste your repository URL here]

Lecturer: Sir Nazmirul Izzad Bin Nassir

Faculty: Faculty of Information Technology, City University Malaysia, Cyberjaya Campus

1. Introduction

For this assignment we were asked to design and build a Java console application that puts Object-Oriented Programming into practice rather than just describing it on paper. The brief specifically calls for classes and objects, encapsulation, inheritance, polymorphism, and abstraction to all show up somewhere in a real, working program, and for the report to explain how and why each one was used. We chose to build a Library Management System, and this report walks through the reasoning behind that choice, how the classes fit together, and how each of the five OOP requirements is actually expressed in the code, not just claimed about it.

We didn't want to just tick boxes on a rubric. Before writing any code we talked through what a library actually needs to keep track of, sketched out which real-world things should become classes, and only then started implementing. The result is a program with nine classes that a librarian can run from the terminal to register members and staff, add books and DVDs to the catalog, lend items out, take them back and work out any late fee, search the catalog, and save everything to a file so nothing is lost when the program is closed. Every part of that workflow is described in the sections that follow, together with the class diagram and the specific lines of code that demonstrate each OOP concept.

2. Problem Description

Picture a small college library with one or two staff on the front desk. A student walks up with a book, the staff member writes the student's name and the book's title into a notebook or an Excel sheet, and moves on to the next person. This works fine until the notebook fills up, someone forgets to write down the date, or two different staff members note the same book as borrowed by two different people because nobody had a single, shared source of truth. Late fees are worse: a DVD that comes back five days late should probably cost more per day than a paperback five days late, but if the fee is worked out by hand it depends entirely on whoever is on duty remembering the right rate.

That's really the core problem we set out to solve: keep track of three things that are always moving – who is registered at the library, what items exist and whether they're currently available, and which member currently holds which item – in a way that doesn't rely on someone's memory or a shared spreadsheet. Object-oriented design turned out to be a natural fit for this, because each of those three concerns maps cleanly onto its own class with its own data and its own rules, instead of one giant function juggling arrays of loosely related values.

Concretely, the console application we built lets a librarian:

- Register new members and new librarians, each keeping their own contact details and role-specific information
- Add new books and DVDs to the catalog, each with the details relevant to that type of item
- Print out a formatted list of every member, librarian, catalog item, or loan currently on record
- Borrow and return items, with the late fee worked out automatically and charged at a different daily rate depending on whether the item is a book or a DVD
- Search the catalog by typing part of a title
- Save the current state of the library to a text file, and load it back in on the next run, so closing the program doesn't wipe out the day's work

3. Class Diagram (UML)

Figure 1 shows all nine classes in the system with their attributes and methods, using standard UML visibility notation (- for private, + for public). The arrows carry meaning too: a hollow triangle points from a subclass up to its superclass (inheritance), an open diamond sits on the side of the class that owns a collection of the other (aggregation), a plain arrow shows one class referring to another (association), and the dashed arrow from Main shows a dependency rather than a stored reference. We laid the diagram out by hand so the two inheritance branches sit clearly apart on the left and right, with Library and Loan bridging them down the middle – it took a couple of attempts to get the lines routed without them crossing through unrelated boxes, but the payoff is that the relationships are actually easy to trace by eye.

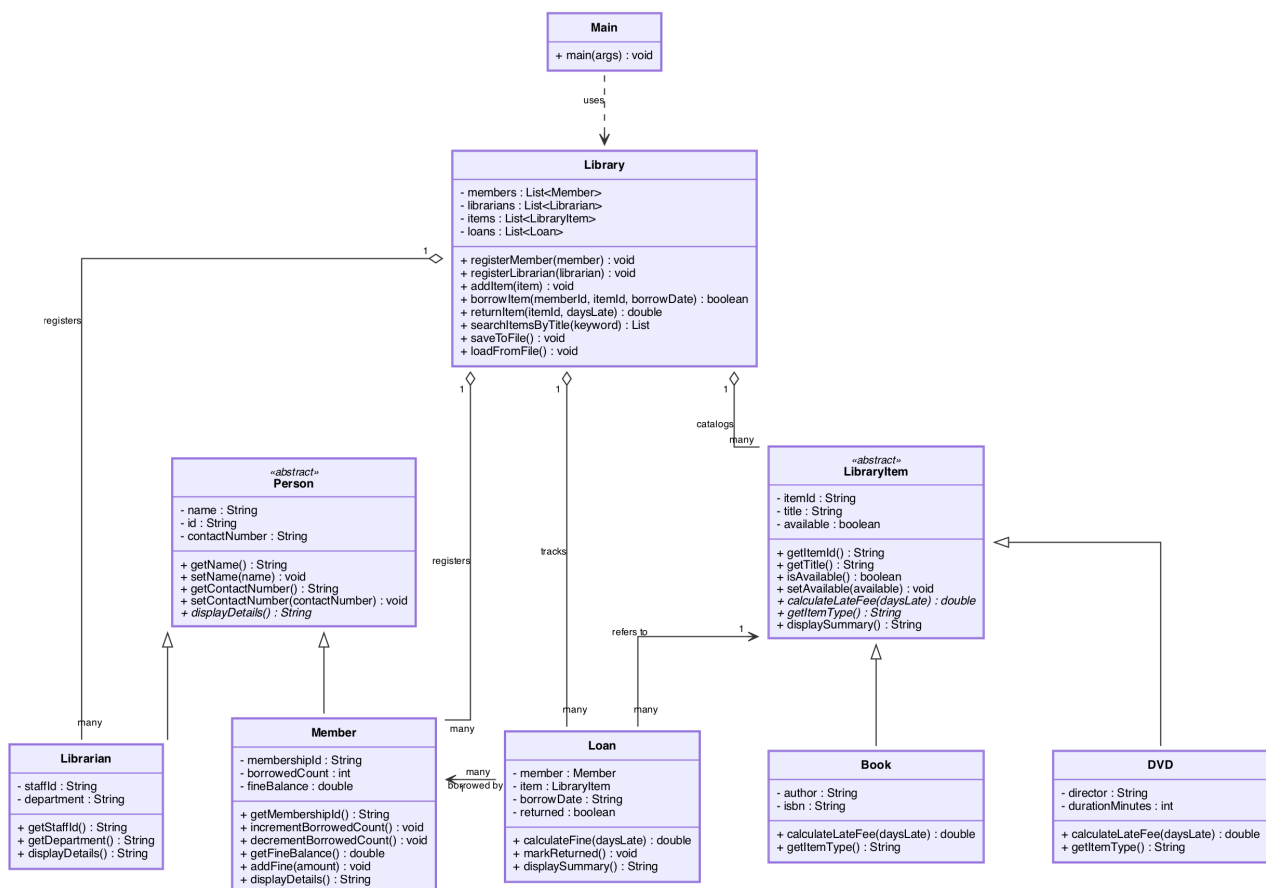


Figure 1: UML Class Diagram of the Library Management System

4. Explanation of UML Design

The design centres on two separate inheritance hierarchies, connected through an association class and managed by a single controller class. None of this was the first idea we had – we went through a couple of rounds of sketching before settling on the structure shown in Figure 1, and it's worth explaining what we tried and rejected along the way, since that reasoning is really the point of a design justification section.

Person → Member / Librarian

Our first instinct was actually a single `Person` class with a `role` field that could be set to “member” or “librarian”, and a pile of if-statements checking which one it was. We dropped that almost immediately because it's exactly the kind of design inheritance exists to replace: a librarian doesn't have a fine balance or a borrowed-item count, and a member doesn't have a department, so cramming both into one class means every object carries fields that don't apply to it. Instead, `Person` holds only what every person in the system genuinely shares – a name, an ID, and a contact number – and is declared abstract because “a person” on its own is never actually registered at the library; only a member or a librarian is. `Member` then adds membership ID, borrowed count, and fine balance, while `Librarian` adds staff ID and department. Each subclass calls `super(...)` to set up the shared fields and then only worries about what makes it different.

LibraryItem → Book / DVD

The same reasoning applies on the catalog side, but with an extra wrinkle: books and DVDs aren't just different in what data they store (author and ISBN versus director and running time), they also need to be charged late fees at different daily rates. That's precisely the situation an abstract method is built for. `LibraryItem` declares `calculateLateFee(daysLate)` and `getItemType()` but deliberately provides no body for either – it can't, because it has no idea what rate applies until you know whether the concrete object is a book or a DVD. `Book` charges RM0.50 per day late; `DVD` charges RM1.00 per day late. Adding a third item type later, say a `Magazine`, would mean writing one new class that extends `LibraryItem` and nothing else in the system would need to change.

Loan as an association class

We also considered just giving `Member` a list of the items it currently has out, and skipping a separate loan class entirely. That falls apart quickly once you think about late fees and history: a fine has to be calculated from a specific borrow date, and a returned loan is still worth keeping a record of, even after the item goes back on the shelf. Bolting all of that onto `Member` would tie it tightly to `LibraryItem` and make the two classes depend on each other directly. So instead we introduced `Loan` as its own class that sits between them, holding a reference to the `Member` who borrowed something, a reference to the `LibraryItem` they borrowed, the date it was borrowed, and whether it's been returned. Neither `Member` nor `LibraryItem` needs to know the other exists – `Loan` is the only class that connects them, which is exactly what an association class is for.

Library as the controller

Finally, something had to actually own the four growing lists of members, librarians, items, and loans, and provide the operations that act on them – registering people, borrowing and returning items, searching, saving and loading. We put all of that in `Library` rather than scattering it across `Main`, so that `Main` only has to worry about reading console input and printing results. That's why the diagram shows `Main` depending on `Library` with a dashed arrow rather than the two being tightly coupled: if we ever swapped the console menu for, say, a simple GUI, `Library` wouldn't need to change at all.

5. Explanation of OOP Concepts Used

This section goes through each of the five OOP requirements one at a time, pointing at the exact class or method where it shows up and, where it makes sense, at what we actually saw happen when we ran the program.

5.1 Classes and Objects

There are nine classes in total: `Person`, `Member`, `Librarian`, `LibraryItem`, `Book`, `DVD`, `Loan`, `Library`, and `Main`, and each one models a single real-world idea rather than being a grab-bag of unrelated data. A class is just the blueprint, though – nothing actually exists until the menu creates an object from it. When a librarian picks “Add new book” from the menu and types in a title and author, `Main` creates one specific `Book` object holding those exact values, completely separate from every other book already in the catalog.

This is what that object creation looks like in `Main.java`:

```
library.registerMember(new Member(name, id, contact, membershipId));
library.addItem(new Book(itemId, title, author, isbn));
```

5.2 Encapsulation

Every field in every class is private. Nothing outside the class can reach in and change an object's data directly – the only way in or out is through the public getter and setter methods the class chooses to expose, and those methods can enforce rules on the way. `Member.addFine()` is a small but telling example: it only accepts a positive amount, so there's no way for a bug elsewhere in the program to accidentally push a member's fine balance negative. If `fineBalance` were a public field, any piece of code anywhere could set it to whatever it wanted, including nonsense values, and we wouldn't be able to guarantee that check ever ran.

```
public double getFineBalance() {
    return fineBalance;
}

public void addFine(double amount) {
    if (amount > 0) {
        this.fineBalance += amount;
    }
}
```

5.3 Inheritance

Both hierarchies discussed in Section 4 use Java's `extends` keyword: `Member` and `Librarian` extend `Person`, and `Book` and `DVD` extend `LibraryItem`. In every case the subclass constructor calls `super( . . . )` first to let the parent class set up the fields it owns, and only then initializes its own additional fields. It's worth pointing out that `Member` never re-declares `name`, `id`, or `contactNumber` – it inherits all three from `Person` and simply reuses `getName()` and the rest without writing a single extra line for them.

```

public class Member extends Person {

    private String membershipId;
    private int borrowedCount;
    private double fineBalance;

    public Member(String name, String id, String contactNumber, String membershipId) {
        super(name, id, contactNumber);
        this.membershipId = membershipId;
        this.borrowedCount = 0;
        this.fineBalance = 0.0;
    }
}

```

5.4 Polymorphism

The clearest example of runtime polymorphism in the project is how a late fee actually gets calculated. `Loan.calculateFine()` never checks whether it's holding a book or a DVD – it just calls `item.calculateLateFee(daysLate)` on whatever `LibraryItem` reference it has, and lets Java figure out at runtime which override actually runs. The same idea shows up again in `Library.displayAllItems()`, which loops over every item using the superclass type and calls `displaySummary()` on each one without caring what's underneath.

```

public double calculateFine(int daysLate) {
    return item.calculateLateFee(daysLate);
}

```

We didn't just take this on faith – we tested it directly while trying out the program. Borrowing a book and returning it three days late charged RM1.50 (three days at RM0.50), while borrowing a DVD and returning it three days late through that exact same `returnItem()` code path charged RM3.00 (three days at RM1.00). Same method call, same loop, two different results, because the object underneath the `LibraryItem` reference was different each time. That's polymorphism actually doing something useful rather than just being a term we can define.

5.5 Abstraction

`Person` and `LibraryItem` are both declared abstract, which means Java won't let anyone write `new Person(...)` or `new LibraryItem(...)` anywhere in the code – only their concrete subclasses can be instantiated. That's a deliberate restriction, not a limitation: on their own, “a person” or “a library item” are too vague to have a working `calculateLateFee()` or a working `displayDetails()`, so the abstract class only defines the shared shape and leaves the actual behaviour for each subclass to fill in.

```
public abstract class LibraryItem {  
    private String itemId;  
    private String title;  
    private boolean available;  
  
    public abstract double calculateLateFee(int daysLate);  
  
    public abstract String getItemType();  
}
```

6. Sample Output

[Insert screenshots of your own terminal run here before submission. A good set to include: the main menu, registering a member, adding a book and a DVD, borrowing an item, returning a book late versus returning a DVD late side by side (so the different fine amounts are visible), and the save/load confirmation messages.]

To compile and run the program from the project root:

```
cd BIT1123-Library-Management-System
javac -d out src/*.java
java -cp out Main
```

7. Conclusion

Working through this assignment made the four pillars of OOP feel a lot less like textbook vocabulary and more like decisions that actually change how easy a program is to extend. The clearest moment of that was realizing how little would need to change if we wanted to add a new kind of item to the catalog: one new class extending `LibraryItem`, and every part of the system that already loops over items – searching, displaying, borrowing, returning – would handle it automatically through polymorphism, without a single if-statement checking “is this a book or a DVD” anywhere in `Library` or `Main`.

The design wasn't perfect on the first attempt, and we don't think it needs to be presented as if it was – the single-class-with-a-role-field idea we tried early on for `Person`, and the version of `Library` that originally didn't persist active loans at all, both got reworked once we actually tested the program end to end and saw where they broke down. If we kept building on this, the next things we'd want to add are due dates that are checked automatically rather than typed in by the librarian, and proper authentication so that only a logged-in librarian account can register new members or add items. For a first pass at applying OOP to a real problem, though, we're happy with how cleanly the final structure maps onto the actual rules of running a library.